CS 320 Project 2

FOOTv8 pipelined processor simulator

Provided files

File / directory	Purpose
`Project2_Instructions.pdf`	Requirements; ISA table on p. 2, pipeline on p. 3
`code/sim.c`, `code/Makefile`	Starting point for your C code and build
`init_mem/`, `pgms/`	Simulator inputs and assembly for reference
`correct_logs/`, `compLog`	Example reports and comparison script
`Makefile`	Build, test, debug, and package commands

What are you building?

Initial-memory file → simulator → pipeline execution → final reports

- Write **C, not C++**; executable: `sim`.
- Take one argument: the initial-memory file path.
- Start fetching at address **0**, in cycle **0**.
- Stop when **EXIT reaches WB**, not when it is fetched.
- Work individually; no collaboration or code sharing.
- Test on `remote.cs.binghamton.edu` or the specified lab machines.

Model the assigned machine

- **32 registers**, X0–X31: 32-bit integers, initially zero.
- **2048 four-byte words** of combined instruction/data memory.
- Load hexadecimal address/value pairs; unlisted memory is zero.
- Instructions are **4 bytes**; immediates are signed **8 or 16 bits**.
- Use the FOOTv8 table provided, not assumptions from full LEGv8.

Input is machine code. You do not need to write an assembler.

Six stages, one clock

IF → DR → EX1 → EX2 → MEM → WB

Stage	Main job
IF	Fetch instruction; advance PC
DR	Decode; read operands
EX1	Add/subtract; begin multiply/divide; compute addresses
EX2	Finish multiply/divide; resolve branches
MEM	Load/store data
WB	Write destination register

Different instructions occupy different stages in the same cycle.

Design around state and next state

State: PC, registers, memory, pipeline boundaries, counters.

Stage logic: reads available state and computes next values.

- Give each pipeline boundary a small struct.
- Carry the instruction's data and needed control information forward.
- Track whether a boundary contains a valid instruction or a bubble.
- Commit boundary updates together at the clock edge.

An instruction must not cross multiple stages in one cycle.

Example: Register component

Two integer fields: **current value** and **future value**.

Operation	Meaning
`setRegisterValue(val)`	Set future value
`getRegisterValue()`	Read current value
`pulseClock()`	Copy future value to current value

current value of `5`

→ `set(9)` → `get()` returns `5`

→ `pulse()` → `get()` returns `9`

Use C functions with a struct; initialize both fields.

Reusable for PC, X0–X31, and inter-stage values.

Timing and hazards matter

- Pulse pipeline registers *together*, not after each stage function.
- **Register-file timing:** WB writes early; DR can read that value in the same cycle (instructions PDF p. 4).
- **No forwarding:** a data hazard stalls IF and DR; older instructions continue, with a bubble behind them.
- **No branch prediction:** a taken branch resolves in EX2; squash IF, DR, and EX1, then redirect the PC.

Plan explicitly for normal pipeline execution, **stalling** IF and DR on data hazards, and **squashing** IF, DR, and EX1 on taken branches.

Test and submit

From the starter project root:

```
make test                       # default: c2f_0
make test TCASE=gcd_33_48       # another provided case
make fullTest                   # all provided input files
make submit                     # creates proj2_code.tar.gz
```

- Final reports start with `=== Simulation Reports ===`.
- Match example formatting: nonzero registers, opcode counts, total instructions, cycles, and IPC.
- Count completed instructions; exclude NOPs and squashed instructions.
- Fix compiler warnings; upload the archive to Brightspace.

First steps

1. Read the PDF; sketch your state and pipeline boundaries.
2. Build the starter; update `code/Makefile` when adding files.
3. Load memory and initialize the PC and registers.
4. Check the register component: reads change only after a pulse.
5. Trace one simple instruction through all six stages.

Then expand instruction support, hazards, and reporting.